

一、基础模板

快读快写

```
// 关闭同步: ios::sync_with_stdio(false); cin.tie(nullptr);
// 换行用 '\n' 而非 endl

// 快读函数 (整数, 替代 scanf, 约快 2~3 倍)
inline int read() {
    int x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9') { if (ch == '-') f = -1; ch = getchar(); }
    while (ch >= '0' && ch <= '9') { x = x * 10 + ch - '0'; ch = getchar(); }
    return x * f;
}

// 快写函数 (整数, 递归实现)
inline void print(int x) {
    if (x < 0) putchar('-'), x = -x;
    if (x > 9) print(x / 10);
    putchar(x % 10 + '0');
}

// 用法: int n = read(); print(ans);
```

二分

整数二分 — 左边界（左边满足，右边不满足）

数组： [F, F, F, T, T, T, T]

索引： 1 2 3 4 5 6 7

↑

分界点（第一个 T）

```
while (l < r) {  
    int mid = (l + r) / 2;  
    if (check(mid)) r = mid;  
    else l = mid + 1;  
}
```

整数二分 — 右边界（左边不满足，右边满足）

适用场景：TTTTFFFF → 找最后一个 T

```
while (l < r) {  
    int mid = (l + r + 1) / 2; // +1 防死循环  
    if (check(mid)) l = mid;  
    else r = mid - 1;  
}
```

浮点数二分

```
while (r - l > 1e-8) { // 也可固定迭代 100 次  
    double mid = (l + r) / 2;  
    if (check(mid)) r = mid;  
    else l = mid;  
}
```

STL 二分

```
int pos = lower_bound(a, a + n, x) - a; // 第一个 >= x  
int pos = upper_bound(a, a + n, x) - a; // 第一个 > x
```

高精度（vector 实现，倒序存：个位在 [0]）

加法

```
vector<int> add(vector<int>& A, vector<int>& B) {  
    vector<int> C;  
    int t = 0;  
    for (int i = 0; i < A.size() || i < B.size(); i++) {  
        if (i < A.size()) t += A[i];  
        if (i < B.size()) t += B[i];  
        C.push_back(t % 10); t /= 10;  
    }  
    if (t) C.push_back(t);  
    return C;  
}
```

减法 (A >= B)

```
vector<int> sub(vector<int>& A, vector<int>& B) {  
    vector<int> C;  
    int t = 0;  
    for (int i = 0; i < A.size(); i++) {  
        t = A[i] - t;  
        if (i < B.size()) t -= B[i];  
        C.push_back((t + 10) % 10);  
        if (t < 0) t = 1; else t = 0;  
    }  
    while (C.size() > 1 && C.back() == 0) C.pop_back();  
    return C;  
}
```

乘低精度

```
vector<int> mul(vector<int>& A, int b) {
    vector<int> C;
    int t = 0;
    for (int i = 0; i < A.size() || t; i++) {
        if (i < A.size()) t += A[i] * b;
        C.push_back(t % 10); t /= 10;
    }
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
```

除低精度

```
vector<int> div(vector<int>& A, int b, int& r) {
    vector<int> C; r = 0;
    for (int i = A.size() - 1; i >= 0; i--) {
        r = r * 10 + A[i];
        C.push_back(r / b); r %= b;
    }
    reverse(C.begin(), C.end());
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
```

前缀和 / 差分

一维前缀和

```
int a[N], pre[N];
// 预处理
for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] + a[i];
// 查询 [l, r] 的和
int sum = pre[r] - pre[l - 1];
```

二维前缀和

```
int a[N][N], s[N][N];
// 预处理
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j];
// 查询 (x1,y1) ~ (x2,y2) 的和
int sum = s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1];
```

一维差分

```
int a[N], d[N]; // d 为差分数组
// 区间 [1, r] 加上 val
d[1] += val;
d[r + 1] -= val;
// 还原原数组
for (int i = 1; i <= n; i++) a[i] = a[i - 1] + d[i];
```

二维差分

```
int d[N][N]; // 差分数组
// 子矩阵 (x1,y1) ~ (x2,y2) 加上 val
d[x1][y1] += val;
d[x2 + 1][y1] -= val;
d[x1][y2 + 1] -= val;
d[x2 + 1][y2 + 1] += val;
// 还原原数组（二维前缀和）
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        d[i][j] += d[i-1][j] + d[i][j-1] - d[i-1][j-1];
// 此时 d[i][j] 即为变化后的 a[i][j]
```

离散化

```
vector<int> v; // 存所有待离散化的值
sort(all(v));
v.erase(unique(all(v)), v.end());
int id = lower_bound(all(v), x) - v.begin() + 1; // 1-indexed
```

二、数论

快速幂

```
ll qpow(ll a, ll b, ll MOD) {
    ll ret = 1;
    while (b) {
        if (b & 1) ret = ret * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return ret;
}
// 求逆元 (MOD 为质数): ll inv(ll a, ll MOD) { return qpow(a, MOD - 2, MOD); }
```

欧几里得

```
ll gcd(ll a, ll b) { return b ? gcd(b, a % b) : a; }
ll lcm(ll a, ll b) { return a / gcd(a, b) * b; } // 先除再乘防溢出
```

扩展欧几里得 (exgcd)

求 $ax + by = \gcd(a, b)$ 的一组整数解。

用途：解二元一次不定方程、求逆元 (MOD 非质数时)、模线性方程。

```

ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1, y = 0; return a; }
    ll d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
// 求 a 在模 m 下的逆元 (gcd(a,m)=1):
// ll x, y; exgcd(a, m, x, y); inv = (x % m + m) % m;

```

求逆元

三种方法，按场景选用。

方法一：快速幂（费马小定理）— 仅 MOD 为质数

```

ll inv(ll a, ll MOD) { return qpow(a, MOD - 2, MOD); }

```

方法二：扩展欧几里得 — MOD 非质数但 gcd=1

```

ll inv(ll a, ll MOD) {
    ll x, y; exgcd(a, MOD, x, y);
    return (x % MOD + MOD) % MOD;
}

```

方法三：线性递推 — 批量求 1~n 所有逆元，O(n)

```

ll inv[N];
void init_inv(int n, ll p) { // p 为质数
    inv[1] = 1;
    for (int i = 2; i <= n; i++)
        inv[i] = p - (p / i) * inv[p % i] % p;
}
// 推导:  $p = k \cdot i + r \rightarrow k \cdot i + r \equiv 0 \rightarrow i^{-1} \equiv -k \cdot r^{-1} \pmod{p}$ 

```

质数

线性筛 $O(n)$

```
int p[N], cnt;
bool st[N];
void get_primes(int n) {
    for (int i = 2; i <= n; i++) {
        if (!st[i]) p[++cnt] = i;
        for (int j = 1; 1ll * i * p[j] <= n; j++) {
            st[i * p[j]] = true;
            if (i % p[j] == 0) break;
        }
    }
}
```

埃氏筛 $O(n \log \log n)$

```
bool st[N];
void get_primes(int n) {
    for (int i = 2; i <= n / i; i++)
        if (!st[i])
            for (int j = i * i; j <= n; j += i) st[j] = true;
}
```

质数判定 $O(\sqrt{n})$

```
bool is_prime(ll x) {
    if (x < 2) return false;
    for (ll i = 2; i * i <= x; i++)
        if (x % i == 0) return false;
    return true;
}
```


组合数（模 MOD 意义下）

```
ll fac[N], invfac[N];
void init_comb(int n) {
    fac[0] = 1;
    for (int i = 1; i <= n; i++) fac[i] = fac[i - 1] * i % MOD;
    invfac[n] = qpow(fac[n], MOD - 2, MOD);
    for (int i = n - 1; i >= 0; i--) invfac[i] = invfac[i + 1] * (i + 1) % MOD;
}
ll C(int n, int m) {
    if (m < 0 || m > n) return 0;
    return fac[n] * invfac[m] % MOD * invfac[n - m] % MOD;
}
// 小范围可用递推: C[i][j] = (C[i-1][j-1] + C[i-1][j]) % MOD;
// n 极大但 MOD 为小质数时用 Lucas: C(n,m) = C(n/MOD,m/MOD) * C(n%MOD,m%MOD) % MOD
```

三、数据结构

并查集 (DSU)

```
int fa[N], sz[N];
int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }
void unite(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return;
    if (sz[a] < sz[b]) swap(a, b);
    fa[b] = a; sz[a] += sz[b];
}
// 初始化: for i=1..n: fa[i]=i, sz[i]=1
```

单调栈

```
// 右边第一个更大的下标（没有则为 0）
int a[N], ans[N];
stack<int> st;
for (int i = 1; i <= n; i++) {
    while (!st.empty() && a[st.top()] < a[i]) // 改成 > 即为更小
        ans[st.top()] = i, st.pop();
    st.push(i);
}
```

单调队列（滑动窗口最值）

```
// 长度为 k 的滑动窗口，求每个窗口的最小值
int a[N];
deque<int> q; // 存下标，保持队内单调递增
for (int i = 1; i <= n; i++) {
    // 1. 弹出队首过期元素
    if (!q.empty() && q.front() < i - k + 1) q.pop_front();
    // 2. 维护单调性（求最小值用 >，求最大值用 <）
    while (!q.empty() && a[q.back()] >= a[i]) q.pop_back();
    q.push_back(i);
    // 3. 取答案（窗口形成后）
    if (i >= k) ans[i - k + 1] = a[q.front()];
}
```

线段树

基础版（单点修改 + 区间求和）

```
ll a[N], tr[N << 2];
void pushup(int u) { tr[u] = tr[u << 1] + tr[u << 1 | 1]; }
void build(int u, int l, int r) {
    if (l == r) { tr[u] = a[l]; return; }
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
    pushup(u);
}
void update(int u, int l, int r, int pos, ll val) {
    if (l == r) { tr[u] += val; return; }
    int mid = (l + r) >> 1;
    if (pos <= mid) update(u << 1, l, mid, pos, val);
    else update(u << 1 | 1, mid + 1, r, pos, val);
    pushup(u);
}
ll query(int u, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tr[u];
    int mid = (l + r) >> 1; ll res = 0;
    if (ql <= mid) res += query(u << 1, l, mid, ql, qr);
    if (qr > mid) res += query(u << 1 | 1, mid + 1, r, ql, qr);
    return res;
}
```

懒标记版（区间加法 + 区间求和）

```
ll a[N], tr[N << 2], lazy[N << 2];
void pushup(int u) { tr[u] = tr[u << 1] + tr[u << 1 | 1]; }
void pushdown(int u, int l, int r) {
    if (lazy[u]) {
        int mid = (l + r) >> 1;
        tr[u << 1] += lazy[u] * (mid - l + 1);
        tr[u << 1 | 1] += lazy[u] * (r - mid);
        lazy[u << 1] += lazy[u];
        lazy[u << 1 | 1] += lazy[u];
        lazy[u] = 0;
    }
}
void build(int u, int l, int r) {
    if (l == r) { tr[u] = a[l]; return; }
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
    pushup(u);
}
void update(int u, int l, int r, int ql, int qr, ll val) {
    if (ql <= l && r <= qr) {
        tr[u] += val * (r - l + 1);
        lazy[u] += val; return;
    }
    pushdown(u, l, r);
    int mid = (l + r) >> 1;
    if (ql <= mid) update(u << 1, l, mid, ql, qr, val);
    if (qr > mid) update(u << 1 | 1, mid + 1, r, ql, qr, val);
    pushup(u);
}
ll query(int u, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tr[u];
    pushdown(u, l, r);
    int mid = (l + r) >> 1; ll res = 0;
    if (ql <= mid) res += query(u << 1, l, mid, ql, qr);
    if (qr > mid) res += query(u << 1 | 1, mid + 1, r, ql, qr);
    return res;
}
// 改成区间最值: pushup 用 max, res 初始 -INF
```

树状数组 (Fenwick Tree / BIT)

比线段树更轻量，仅支持**单点加 + 前缀和**（可通过差分扩展为区间加）。代码量少、常数极小。

```
ll tr[N]; // 树状数组，下标从 1 开始

int lowbit(int x) { return x & -x; }

// 单点加: a[pos] += val
void add(int pos, ll val, int n) {
    for (; pos <= n; pos += lowbit(pos))
        tr[pos] += val;
}

// 前缀和: a[1] + ... + a[pos]
ll sum(int pos) {
    ll res = 0;
    for (; pos > 0; pos -= lowbit(pos))
        res += tr[pos];
    return res;
}

// 区间查询 [l, r]
ll query(int l, int r) { return sum(r) - sum(l - 1); }
```

注意：树状数组天然支持单点修改 + 区间求和。若需区间修改 + 单点查询，对差分数组建 BIT；若需区间修改 + 区间查询，维护两个 BIT。

ST 表（静态 RMQ）

```
int f[N][20], Log2[N];
void init_st(int a[], int n) {
    Log2[0] = -1;
    for (int i = 1; i <= n; i++) {
        f[i][0] = a[i];
        Log2[i] = Log2[i >> 1] + 1;
    }
    for (int j = 1; (1 << j) <= n; j++)
        for (int i = 1; i + (1 << j) - 1 <= n; i++)
            f[i][j] = max(f[i][j - 1], f[i + (1 << (j - 1))][j - 1]);
}
int query_st(int l, int r) {
    int k = Log2[r - l + 1];
    return max(f[l][k], f[r - (1 << k) + 1][k]);
}
```

字符串哈希

将字符串映射为一个整数（哈希值），从而在 $O(1)$ 内判断两个子串是否相等。

常用场景：判断子串相等、最长公共前缀(LCP)、回文串判定、字符串匹配（替代 KMP）。

```

typedef unsigned long long ull;
const ull P = 131;          // 基数，也可用 13331
ull h[N], p[N];             // h[i]: 前缀 s[1..i] 的哈希, p[i]:  $P^i$ 

void init_hash(string& s, int n) {
    p[0] = 1;
    for (int i = 1; i <= n; i++) {
        h[i] = h[i - 1] * P + s[i - 1]; // 看作 P 进制数
        p[i] = p[i - 1] * P;
    }
}

// 子串 s[l..r] 的哈希值 (1-indexed)
ull get_hash(int l, int r) {
    return h[r] - h[l - 1] * p[r - l + 1];
}

// 判断两个子串是否相等
bool same(int l1, int r1, int l2, int r2) {
    return get_hash(l1, r1) == get_hash(l2, r2);
}

// 求两个后缀的最长公共前缀长度 (二分 + 哈希)
int lcp(int a, int b) { // 后缀 a 和 b 的 LCP
    int l = 0, r = min(n - a + 1, n - b + 1);
    while (l < r) {
        int mid = (l + r + 1) / 2;
        if (get_hash(a, a + mid - 1) == get_hash(b, b + mid - 1))
            l = mid;
        else r = mid - 1;
    }
    return l;
}

```

注意：单哈希可能被卡，必要时用双哈希（两对不同 P 和 MOD ）或使用 `unsigned long long` 自然溢出（等效模 2^{64} ）。

四、图论

最短路

Dijkstra（堆优化，正权图， $O(m \log n)$ ）

```
vector<PII> g[N]; // {v, w}
int dist[N]; bool vis[N];
void dijkstra(int s) {
    memset(dist, 0x3f, sizeof dist);
    priority_queue<PII, vector<PII>, greater<PII>> pq;
    dist[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (vis[u]) continue; vis[u] = true;
        for (auto [v, w] : g[u])
            if (dist[v] > d + w)
                dist[v] = d + w, pq.push({dist[v], v});
    }
}
```

BFS（无权图， $O(n+m)$ ）

```
vector<int> g[N]; int dist[N];
void bfs(int s) {
    memset(dist, -1, sizeof dist);
    queue<int> q;
    dist[s] = 0; q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u])
            if (dist[v] == -1)
                dist[v] = dist[u] + 1, q.push(v);
    }
}
```


SPFA (可负权, 判负环, $O(km)$)

```
vector<PII> g[N];
int dist[N], cnt[N]; bool inq[N];
bool spfa(int s, int n) { // true=无负环
    memset(dist, 0x3f, sizeof dist);
    queue<int> q;
    dist[s] = 0; q.push(s); inq[s] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop(); inq[u] = false;
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                cnt[v] = cnt[u] + 1;
                if (cnt[v] >= n) return false;
                if (!inq[v]) q.push(v), inq[v] = true;
            }
        }
    }
    return true;
}
```

Floyd (全源, $O(n^3)$)

```
for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            f[i][j] = min(f[i][j], f[i][k] + f[k][j]);
// 初始化: f[i][i]=0, 有边 f[u][v]=w, 其他 INF
```

拓扑排序 (Kahn, $O(n+m)$)

```
vector<int> g[N]; int in[N];
vector<int> topo_sort(int n) {
    vector<int> res; queue<int> q;
    for (int i = 1; i <= n; i++)
        if (!in[i]) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        res.pb(u);
        for (int v : g[u])
            if (--in[v] == 0) q.push(v);
    }
    return res; // size() < n 则有环
}
```

倍增求 LCA

```
vector<int> g[N]; int dep[N], fa[N][20];
void dfs(int u, int father) {
    dep[u] = dep[father] + 1;
    fa[u][0] = father;
    for (int i = 1; (1 << i) <= dep[u]; i++)
        fa[u][i] = fa[fa[u][i-1]][i-1];
    for (int v : g[u])
        if (v != father) dfs(v, u);
}
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int d = dep[u] - dep[v];
    for (int i = 0; d; i++, d >>= 1)
        if (d & 1) u = fa[u][i];
    if (u == v) return u;
    for (int i = 19; i >= 0; i--)
        if (fa[u][i] != fa[v][i])
            u = fa[u][i], v = fa[v][i];
    return fa[u][0];
}
// 树上第 k 祖先: for(i=0;k;i++,k>>=1) if(k&1) u=fa[u][i];
```

Kruskal 最小生成树 ($O(m \log m)$)

```
struct Edge { int u, v, w; } e[M]; // M 为边数
bool cmp(Edge a, Edge b) { return a.w < b.w; }

ll kruskal(int n, int m) { // 返回 MST 总权值, 不连通返回 -1
    sort(e + 1, e + 1 + m, cmp);
    // 并查集初始化: for i=1..n: fa[i]=i
    ll res = 0; int cnt = 0;
    for (int i = 1; i <= m; i++) {
        int u = find(e[i].u), v = find(e[i].v);
        if (u != v) {
            fa[u] = v;
            res += e[i].w;
            if (++cnt == n - 1) return res;
        }
    }
    return -1; // 不连通
}
```

五、动态规划

背包 DP

$dp[i][j]$ 表示前 i 件物品、容量为 j 时的最大价值。

01 背包（每件物品最多选一次）

```
// 二维原版:  $dp[i][j] = \max(dp[i-1][j], dp[i-1][j - w[i]] + v[i])$ 
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++) {
        dp[i][j] = dp[i - 1][j]; // 不选第 i 件
        if (j >= w[i])
            dp[i][j] = max(dp[i][j], dp[i - 1][j - w[i]] + v[i]); // 选第 i 件
    }
// 答案:  $dp[n][m]$ 
```

```
// 优化版（一维，倒序）：dp[j] 等价于 dp[i-1][j]，倒序防止物品被重复选
for (int i = 1; i <= n; i++)
    for (int j = m; j >= w[i]; j--)
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
```

完全背包（每件物品无限次）

```
// 二维原版：dp[i][j] = max(dp[i-1][j], dp[i][j - w[i]] + v[i])
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++) {
        dp[i][j] = dp[i - 1][j]; // 不选第 i 件
        if (j >= w[i])
            dp[i][j] = max(dp[i][j], dp[i][j - w[i]] + v[i]); // 选第 i 件（还可再选）
    }
// 答案：dp[n][m]
```

```
// 优化版（一维，正序）：正序让 dp[j-w[i]] 已包含本轮更新，等价于 dp[i][j-w[i]]
for (int i = 1; i <= n; i++)
    for (int j = w[i]; j <= m; j++)
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
```

多重背包（第 i 件最多 s[i] 次）

```
// 朴素二维：枚举选 k 件
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        for (int k = 0; k <= s[i] && k * w[i] <= j; k++)
            dp[i][j] = max(dp[i][j], dp[i - 1][j - k * w[i]] + k * v[i]);
```

```
// 优化版：二进制拆分 → 01 背包（一维倒序）
for (int i = 1; i <= n; i++) {
    int vv = v[i], ww = w[i], k = s[i];
    for (int t = 1; t <= k; t <<= 1) {
        for (int j = m; j >= t * ww; j--)
            dp[j] = max(dp[j], dp[j - t * ww] + t * vv);
        k -= t;
    }
    if (k) for (int j = m; j >= k * ww; j--)
        dp[j] = max(dp[j], dp[j - k * ww] + k * vv);
}
```